

DDTA - Working Metamodel - Macro Requirement

DOCUMENTATION-LAYER CLOSURE CANDIDATE - REVISION 8

This revision consolidates the Macro Requirement semantics after the Functional Requirement study. It keeps ThreatForge-specific observations outside the general metamodel and introduces project problem framing as a method precondition, not as a new governed document type.

1. Scope and layering

The conceptual model describes governed project knowledge before fixing concrete Mark-down/YAML encoding or tool behavior. The Doc-as-Code representation remains versionable, diffable and reviewable; a tool may validate or assist the model but does not own project semantics.

The layers remain:

1. **L1 conceptual metamodel** - meaning, concepts, relations, cardinalities and invariants;
2. **L2 canonical representation** - concrete fields, headings, registries and encoding;
3. **L3 realization principles** - resolution, validation, projections and authoring assistance;
4. **L4 tool support/conformance** - which contracts a tool supports.

2. Project problem framing - required method precondition

Before decomposing Macro Requirements, the author states the general problem or class of problems addressed by the project and its meaningful boundary, temporarily removing current solution choices.

Fundamental framing question

Which general problem or class of problems must the project address, within which meaningful boundary, if current solution choices are temporarily removed?

The framing:

- uses problem/domain vocabulary rather than current architecture vocabulary;
- prevents chosen devices, providers, algorithms, databases or analysis methods from becoming accidental MR semantics;
- is used to test completeness, overlap and stability of the MR set;
- is **not yet** a ProjectProblem or ProjectContext governed-document type.

The framing is written once for the project. It is not copied into every MR.

3. Progressive governed-document hierarchy

For the thesis baseline, the governed authoring hierarchy is intentionally regular:

MacroRequirement

```
|
+-- Decision *
    |
    +-- FunctionalRequirement *
        |
        +-- SpecializedRequirement *
```

The hierarchy is semantic containment, not identifier-string parsing or folder inference.

- MR establishes one stable macro responsibility within the framed problem.
- Decision records the governed commitment that narrows that responsibility.
- FR states the operational behavior required under that commitment.
- Specialized Requirements constrain/enrich an FR. For the thesis implementation/evaluation path, the next specialization to formalize is SecurityRequirement.

No Decision is a child of an FR. If a later choice changes the design, it remains a Decision under the relevant MR and the affected FR branch is revised/superseded accordingly.

4. Macro Requirement definition

A Macro Requirement is a governed root document that owns one durable macro responsibility or concern needed to address the framed project problem. It explains the macro purpose/value, minimum context, affected stakeholders and semantic boundary without selecting lower-level solution commitments or stating independently testable functional behavior.

Fundamental MR question

Within the framed project problem, which stable macro responsibility or concern are we governing, what macro result/value must it contribute, why does it matter, and who is involved?

5. Semantic shape

Concept	State	Purpose
id	required	stable identity, history and traceability
title	required	concise name understandable within the project domain
lifecycle	required	current/historical governance state
intent	required	macro purpose, value and desired outcome
context	required	minimum background needed to interpret the concern
stakeholders	required	people/groups that benefit, use, govern or are affected
scope	semantically required	boundary of the macro concern; exact L2 encoding remains rep
assumptions	optional	facts treated as true for the whole branch
constraints	optional	limits applying to the whole branch
dependsOn	MacroRequirement [0..*]	non-hierarchical dependency on another macro responsibility

Objective remains removed. Intent owns macro purpose, value and desired outcome.

6. MR authoring questions

1. What general problem/class did the project framing establish?
2. Which durable responsibility inside that problem does this MR own?
3. Can a domain-competent stakeholder understand title + Intent without knowing the current implementation?
4. Does the MR remain valid if lower-level providers, devices, algorithms, deployment or architecture change?
5. Does Context contain only information required to interpret the macro concern rather than the current design state?
6. Who benefits, governs, uses or is materially affected?
7. What is inside/outside this MR's responsibility boundary?
8. Is another MR genuinely complementary/dependent? If yes, use dependsOn, not false hierarchy or duplicate ownership.
9. Has any significant solution choice leaked into MR prose? If so, move it to Decision.

10. Has any independently assessable behavior leaked into MR prose? If so, move it downstream to an FR under a Decision.

7. Closed MR invariants for the documentation baseline

1. **Root hierarchy** - MR has no hierarchical parent.
2. **Stable identity** - identity is independent from textual/composite encoding.
3. **Problem anchoring** - every MR is meaningful inside the project problem framing.
4. **Single macro concern** - one coherent macro responsibility per MR.
5. **Child narrowing** - Decisions and their FR descendants narrow the MR rather than introduce unrelated scope.
6. **No solution commitment leakage** - chosen architecture/provider/algorithm/mechanism belongs to Decision unless intrinsic to the problem domain itself.
7. **No FR-level obligation** - atomic/operational independently assessable behavior belongs downstream.
8. **Broad domain readability** - MR can be understood by stakeholders competent in the problem domain.
9. **Architecture resilience** - changing lower-level solution choices should not normally require rewriting the MR.
10. **Temporal stability** - taking/revising a lower-level Decision should not make MR prose obsolete unless the macro concern changes.
11. **Explicit dependencies** - cross-MR complementarity/dependency is not hidden as pseudo-hierarchy.
12. **Teachability** - titles + Intent across the MR set support a useful macro project map.
13. **Analysis enabling, not analysis producing** - MR provides governed context but does not instantiate canonical analysis elements or method-specific semantics.
14. **Projection honesty** - a teaching MR map is not a DFD, architecture model or threat model.

8. MR split/merge tests

Before splitting or merging candidate MRs, test:

- **independent value** - does each candidate express a project-level result meaningful on its own?
- **Decision-family coherence** - do its descendant commitments form a coherent family?
- **solution-vocabulary removal** - does the distinction survive after removing current solution names?
- **dependency versus containment** - can the relation be expressed more accurately as dependsOn?
- **stakeholder explanation** - can a domain stakeholder explain the distinction from title + Intent?
- **merge stress** - would merging force substantially different responsibilities/Decision families into one MR?
- **split stress** - is separation driven by a project responsibility rather than an architectural subsystem?

9. Analysis boundary

MRs may later support Base Analysis construction through governed downstream documentation, but they do not directly create canonical BAE, flows, boundaries or method-specific interpretations merely because nouns appear in MR prose.

The next research phase will formalize Base Analysis/BAE and overlay behavior without changing this writing model unless a concrete counterexample requires reopening it.

10. Representation deliberately deferred

L2 still decides:

- where MR concepts live between registry and Markdown;
- exact scope syntax;
- whether parent/null values are materialized;
- concrete depends on encoding;
- any identifier format that mirrors hierarchy.

Identity/topology must remain semantic relations rather than consequences of filenames or composite IDs.